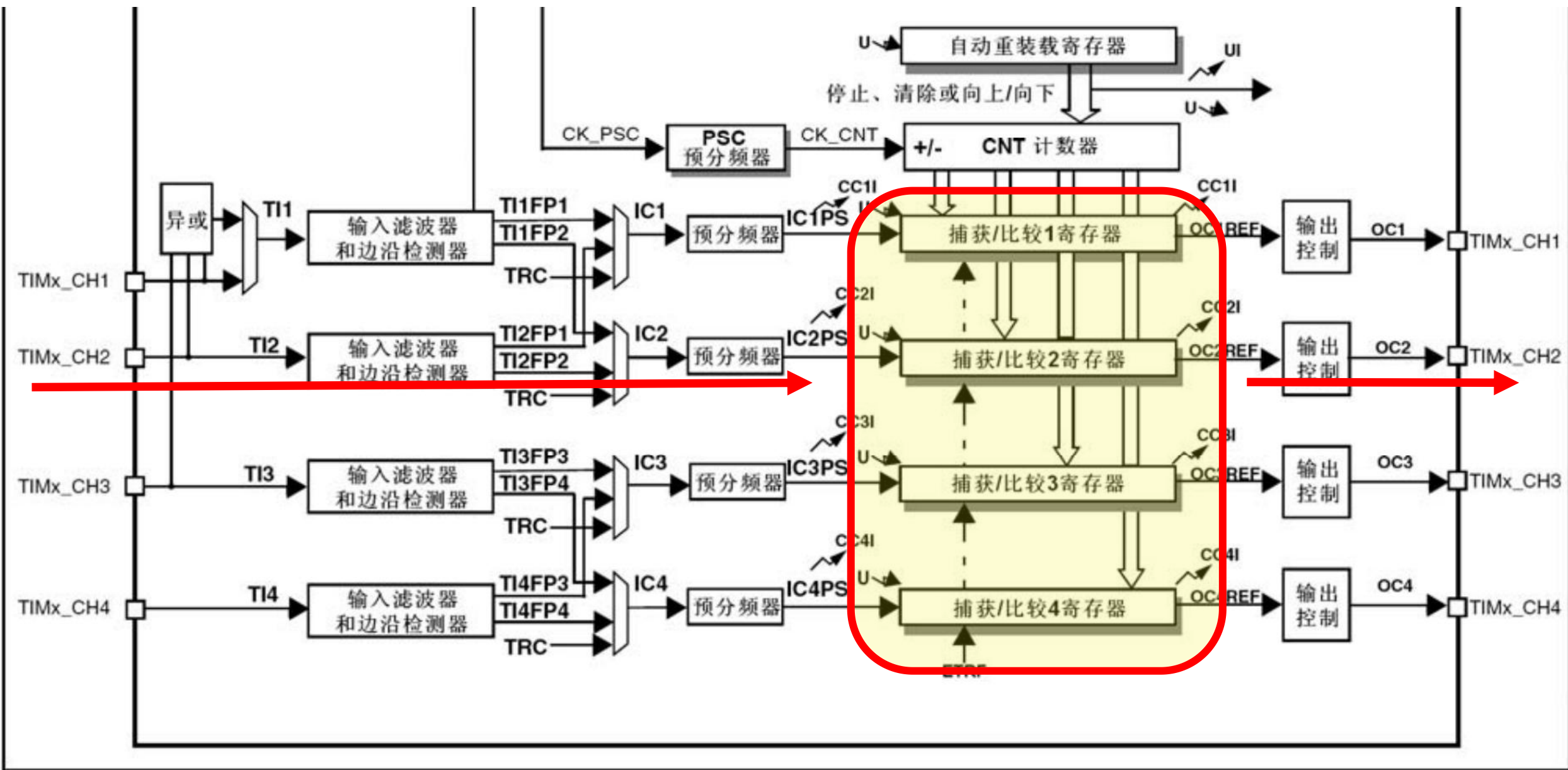


第七讲 stm32单片机的通用定时器

- ◆ STM32定时器的引脚
- ◆ 输出比较的概念
- ◆ 定时器
- ◆ 引脚反转
- ◆ PWM模式



一、STM32单片机的TIM通道及引脚

- ★ TIM1为高级定时器，含有互补通道3对，普通通道1个。
- ★ TIM2、3、4为普通定时器，含有通道4个。
- ★ 一个定时器本身内部有一个计数器 CNT 在跑。
- ★ 而“channel”就是让这个计数值和外部GPIO引脚发生联系的功能单元。
- ★ 即是TIM与外部引脚打交道的接口，主要有2大功能：

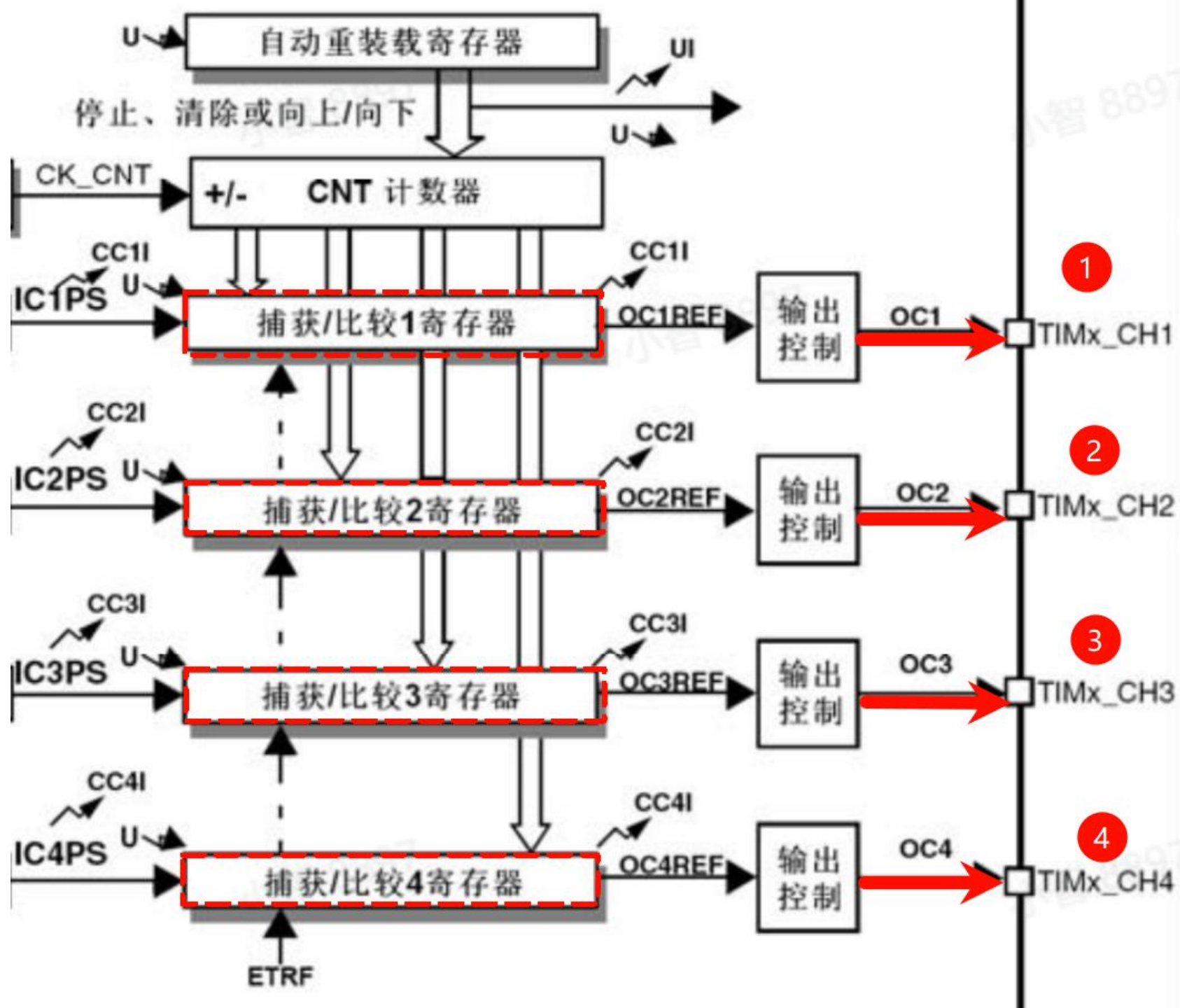
1、输出功能

- ★ 比如输出 PWM 波
- ★ 输出比较匹配信号

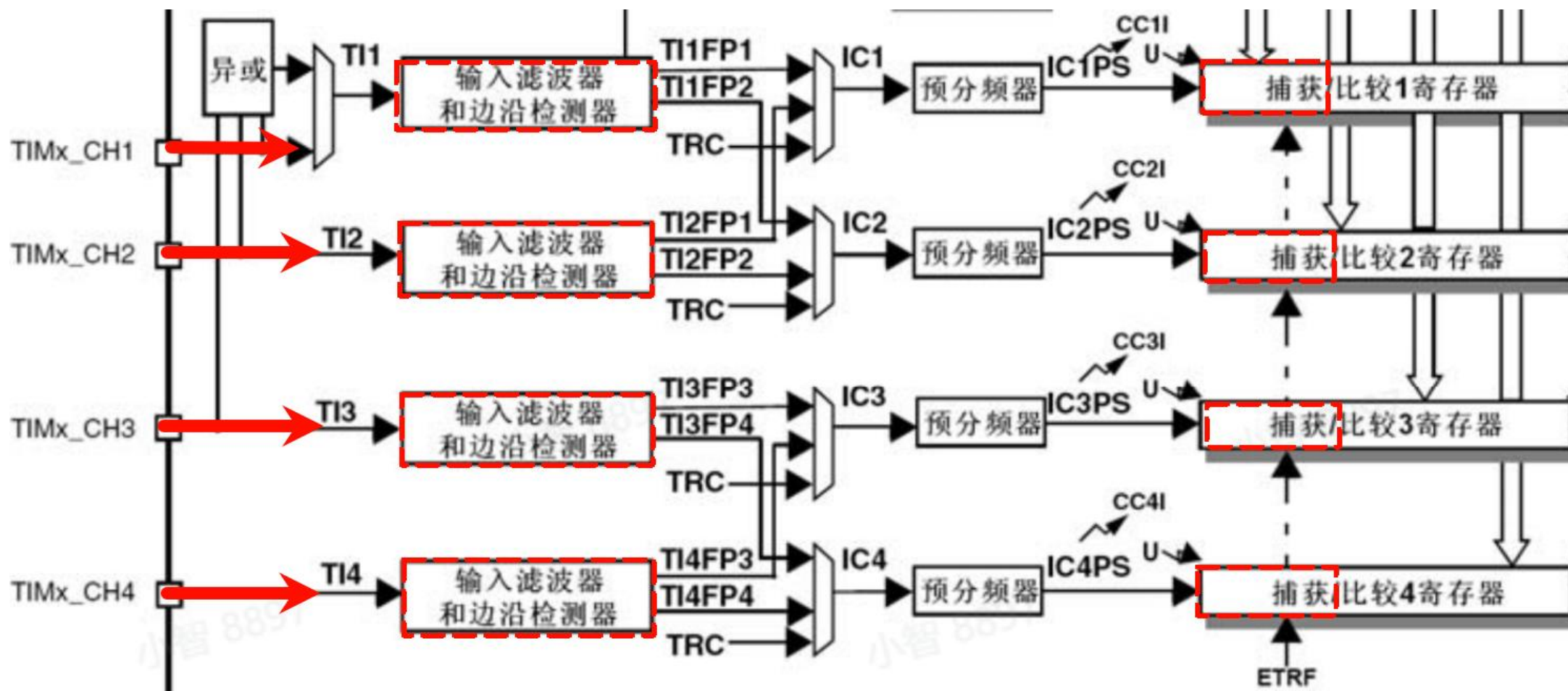
2、输入功能

- ★ 比如检测外部脉冲来到的时刻
- ★ 测频率、测脉宽。

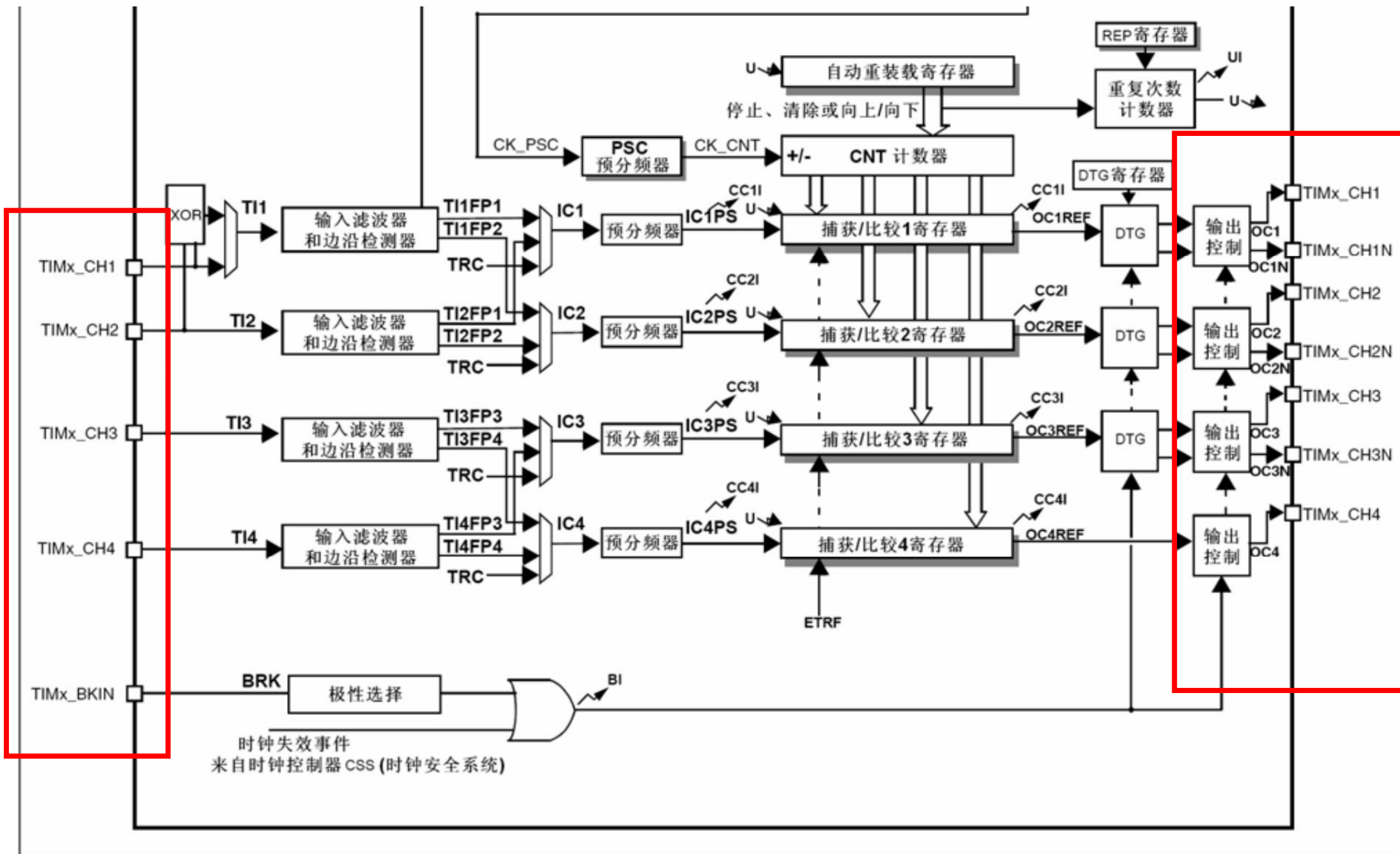
4路输出比较：
用于输出PWM
波形、方波、驱
动电机等。



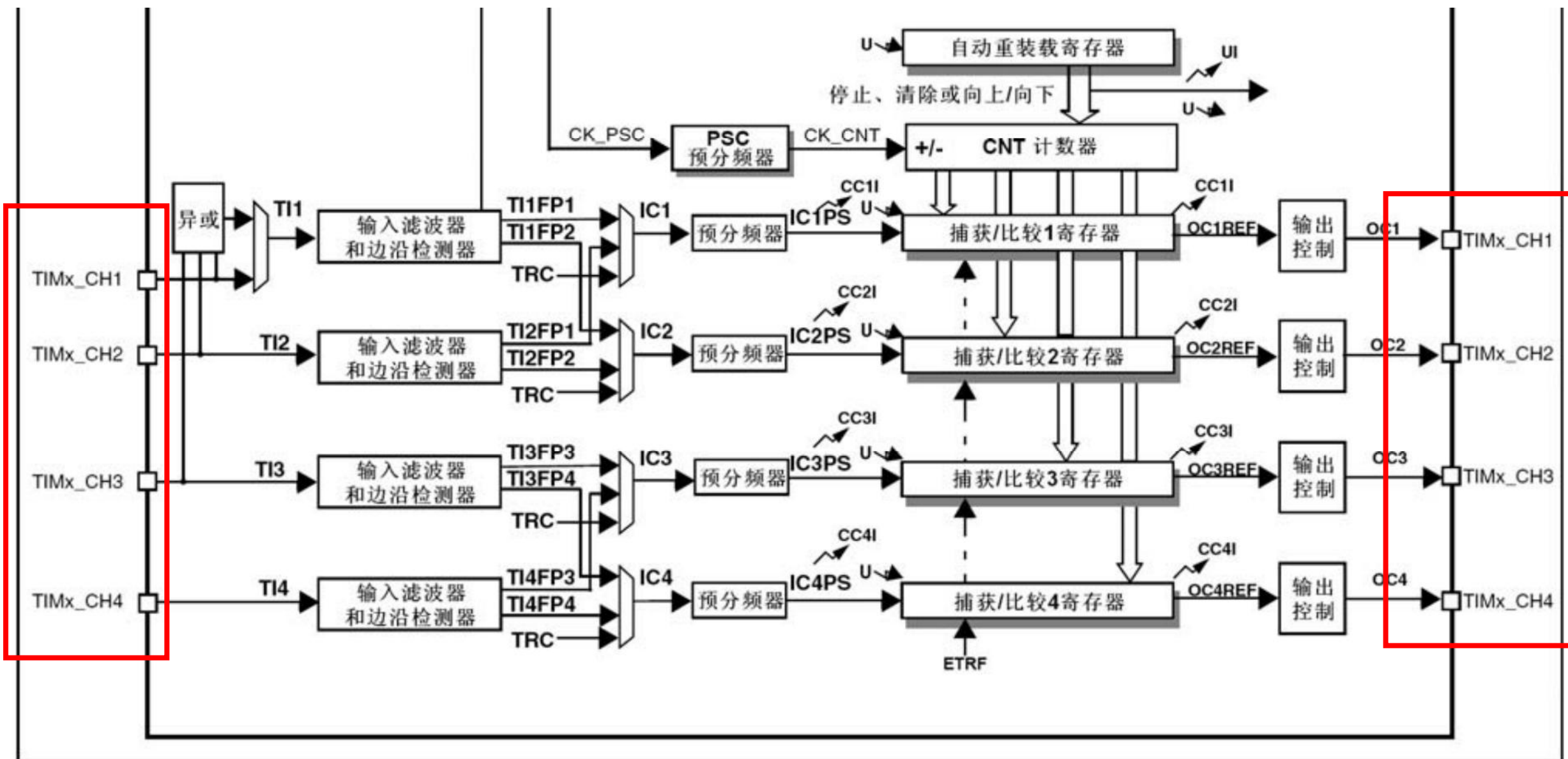
4路输入捕获，用于测量输入信号的周期、脉宽等。



TIM1



TIM2、3、4



一、STM32单片机的TIM通道及引脚

定时器	默认映射GPIO (CH1、2、3、4)	部分重映射1	部分重映射2	完全重映射
TIM1	PA8(PB13) PA9(PB14)、 PA10(PB15)、 PA11	PA8(PA7)、 PA9(PB0)、 PA10(PB1)、 PA11		
TIM2	PA0、PA1、PA2、 PA3	PA15、PB3、PA2、 PA3	PA0、PA1、 PB10、PB11	PA15、PB3、 PB10、PB11
TIM3	PA6、PA7、PB0、 PB1	PB4、PB5、PB0、 PB1		PC6、PC7、 PC8、PC9
TIM4	PB6、PB7、PB8、 PB9			

STM32F103RBT6 的 TIM1 是高级定时器，它的 4 个主通道部分映射不变，主要改变的是**互补输出通道和刹车输入引脚**：

TIM1 通道	默认映射	部分重映射
TIM1_CH1	PA8	PA8
TIM1_CH2	PA9	PA9
TIM1_CH3	PA10	PA10
TIM1_CH4	PA11	PA11
TIM1_BKIN	PB12	PA6
TIM1_CH1N	PB13	PA7
TIM1_CH2N	PB14	PB0
TIM1_CH3N	PB15	PB1

重映射(Remap):

同一个通道可以映射到不同的引脚上，意思是把某个外设功能从**默认**引脚**改到****另一组**预定义引脚上。

为什么要重映射?

- 因为芯片引脚资源很紧张，一个引脚往往有多个复用功能。
- 比如一个引脚可能同时可以做：GPIO、USART、SPI、TIM 通道
- 如果默认引脚已经被别的外设占用了，就需要把 TIM 的通道换到别的引脚上。
- 重映射的意义：避免引脚冲突、方便 PCB 布线、满足实际硬件设计需求

一、STM32单片机的TIM通道及引脚

★定时器，选用哪一组引脚，由重映射函数来确定：

如： `GPIO_PinRemapConfig(GPIO_PartialRemap1_TIM2, ENABLE)` ;
使能 TIM2 的部分重映射方案1

告诉芯片：TIM2 不再用默认那组引脚，改成“部分重映射1”对应的引脚组合

★使用引脚重映射功能，还应给引脚复用功能开时钟：

`RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);`

一、STM32单片机的TIM通道及引脚

- ★ 引脚应初始化成**GPIO_Mode_AF_PP**模式（复用推挽）或**GPIO_Mode_AF_OD**（复用漏开），具体用哪个，根据后面电路来确定。
- ★ 每个通道有一个16位的**捕获/比较寄存器**。它用来和**计数器**的值进行**比较或存放**计数器的当前值。

二、TIM的输出比较

定时器的输出比较通道（OC通道）有6种常见输出模式，分别为：

（1）定时器模式（Timing），输出引脚不发生变化，只用作定时器触发中断事件。

（2）主动输出模式，比较匹配，对应引脚为高电平，使用场景较少。

（3）被动输出模式，比较匹配，对应引脚为低电平，使用场景较少。

（4）翻转（Toggle）模式，比较匹配，引脚反转，可用于各种频率方波信号的产生。

（5）PWM1模式，加法计数且比较成功时，引脚置低电平，CNT溢出时，引脚置高（翻转2次，所以可以实现PWM）。

（6）PWM2模式，加法计数且比较成功时，引脚置高电平，溢出时，引脚置低。

二、TIM的输出比较

```
#define TIM_OCMode_Timing                ((uint16_t) 0x0000)
#define TIM_OCMode_Active                ((uint16_t) 0x0010)
#define TIM_OCMode_Inactive              ((uint16_t) 0x0020)
#define TIM_OCMode_Toggle                ((uint16_t) 0x0030)
#define TIM_OCMode_PWM1                  ((uint16_t) 0x0060)
#define TIM_OCMode_PWM2                  ((uint16_t) 0x0070)
```

2.1 定时器模式

工作流程：

- (1) 要根据情况，设置预分频系数，计数器的周期一般设为65535。
- (2) 设置比较值，该值为定时器所要求的周期数值。该值设在比较/捕获寄存器中。当该值与计数器的值，相同时，比较成功，触发中断。
- (3) 设置中断优先级，中断源为比较/捕获，不使用溢出。
- (4) 写中断服务函数，在中断服务函数中，获取当前的计数器的值，将定时周期累加到比较/捕捉寄存器中。
- (5) 中断服务函数中，应分清是哪个通道引起的中断。

2.1 定时器模式

如：配置 TIM4 的通道 4 为**输出比较定时模式**，当 TIM4 计数到 10 时产生 CC4 比较中断。

```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_Timing;  
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Disable;  
TIM_OCInitStructure.TIM_Pulse = 10;  
TIM_OC4Init(TIM4, &TIM_OCInitStructure);  
  
TIM_Cmd(TIM4, ENABLE);  
TIM_ITConfig(TIM4, TIM_IT_CC4, ENABLE);
```

```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_Timing;
```

配置 TIM4 通道 4 的输出比较模式为 Timing 模式。只做“计数比较”，不控制引脚输出 PWM 或翻转电平。也就是说，TIM4 的 CNT 会一直计数，当：CNT == CCR4时，会产生一个比较事件，可以触发中断。

```
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Disable;
```

关闭通道 4 的输出功能，TIM4_CH4 对应的 GPIO 引脚不会输出波形。所以这段代码不是用来产生 PWM 的，而是单纯用来产生定时中断。

```
TIM_OCInitStructure.TIM_Pulse = 10;
```

设置比较值 CCR4 = 10，当 TIM4 的计数器 CNT 计数到 10 时，会触发 CC4 比较事件。如果让它严格每 10 个脉冲触发一次，通常需要在中断服务函数里动态更新 CCR4：
TIM_SetCompare4(TIM4, TIM_GetCapture4(TIM4) + 10);

```
TIM_OC4Init(TIM4, &TIM_OCInitStructure);
```

把前面配置的参数写入 TIM4_CH4

```
TIM_Cmd(TIM4, ENABLE);
```

启动 TIM4 定时器，执行后，TIM4 的计数器 CNT 开始计数。

```
TIM_ITConfig(TIM4, TIM_IT_CC4, ENABLE);
```

使能 TIM4 的 **通道 4 比较中断**，即允许 TIM4 在发生 CC4 比较事件时产生中断。也就是：CNT == 10时，会置位 CC4IF 标志位，并触发 TIM4 中断。

例程：基于 TIM1 多通道（4个通道）输出比较中断的电子 时钟系统 -----输出比较定时模式应用

- 定时器基础计数周期为 0.1ms，通过设置 CCR1~CCR4 分别在 1ms、2ms、100ms 和 1s 产生比较中断。
- 在中断服务函数中：
 - CC1 用于数码管动态扫描显示；
 - CC2 用于矩阵键盘扫描手动调整时分秒和时钟数据更新；
 - CC3 用于 LED 流水灯控制；
 - CC4 用于秒计数。

整个程序体现了“一个定时器，多通道，多任务调度”的思想。

练习：修改流水灯速度

修改矩阵按键调整时间

四个通道的任务分工：

通道	比较增量	中断周期	任务
TIM1_CC1	+10	$10 \times 0.1\text{ms} = 1\text{ms}$	数码管动态扫描
TIM1_CC2	+20	$20 \times 0.1\text{ms} = 2\text{ms}$	按键扫描/更新时间
TIM1_CC3	+1000	$1000 \times 0.1\text{ms} = 100\text{ms}$	LED流水灯
TIM1_CC4	+10000	$10000 \times 0.1\text{ms} = 1\text{s}$	秒计数+1

系统上电/复位

Init_All_Periph() RCC / GPIO / TIM1 / NVIC

启动 TIM1 计数, PSC=7199, ARR=65535, 计数周期 = 0.1ms

TIM1 计数器不断递增 CNT: 0 → 65535

CNT == CCR1 ?
(10,20...)

是, 置位 CC1IF

CNT == CCR2 ?
(20,40...)

是, 置位 CC2IF

CNT == CCR3 ?
(1000,2000...)

是, 置位 CC3IF

CNT == CCR4 ?
(10000,20000...)

是, 置位 CC4IF

进入 TIM1_CC_IRQHandler

2.2 引脚反转（Toggle）模式

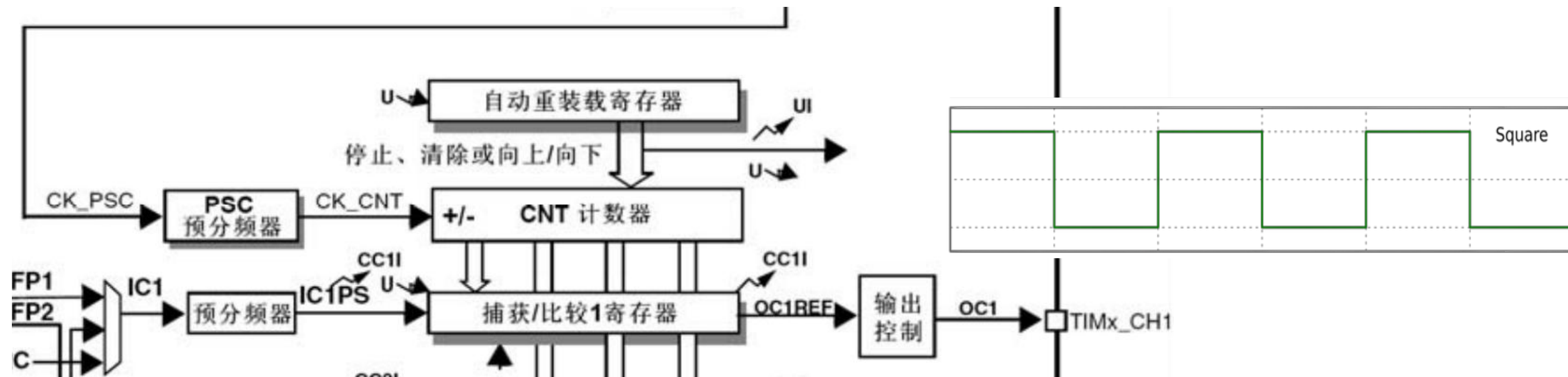
工作流程：

- 该方法与定时器模式相比，增加了引脚**电平反转** 功能。
- 当预先存在比较/捕获寄存器的值，与计数器的值，相同时，比较成功，触发中断和触发引脚电平翻转。
- 因为使用引脚，需要将引脚设置成**复用开漏或推挽**，同时根据引脚情况，用**重映射函数**来确定引脚，用GPIO**复用时钟函数**来使重映射有效。
- 利用该功能，**可以高效地产生各种频率的方波**

2.2 引脚反转 (Toggle) 模式

```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_Toggle;  
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;  
TIM_OCInitStructure.TIM_Pulse = 500;  
TIM_OC1Init(TIM4, &TIM_OCInitStructure);  
TIM_ITConfig(TIM4, TIM_IT_CC1, ENABLE);
```

```
x = TIM_GetCapture1(TIM4) + 500;  
TIM_SetCompare1(TIM4, x); // 下一次比较值 = 当前比较值 + x
```



例程：基于 STM32 TIM4 输出比较模式的多路方波发生器设计

翻转 (Toggle) 模式 (比较匹配, 引脚反转)

- 1、使用 TIM4 的 4 个输出比较通道 CH1~CH4，在 PB6、PB7、PB8、PB9 四个引脚上输出不同周期的方波
- 2、方波信号的频率以led的闪烁快慢来展示。
- 3、通过**按键调节** TIM4_CH1 输出方波的周期，并用数码管显示当前周期参数。

TIM4

PB6、PB7、PB8、PB9

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7 | GPIO_Pin_8 | GPIO_Pin_9;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

例程：基于 STM32 TIM3 输出比较模式的矩阵键盘简易电子琴设计

用 STM32 的矩阵键盘控制蜂鸣器发出不同音调，并在数码管上显示当前按键编号和对应频率。

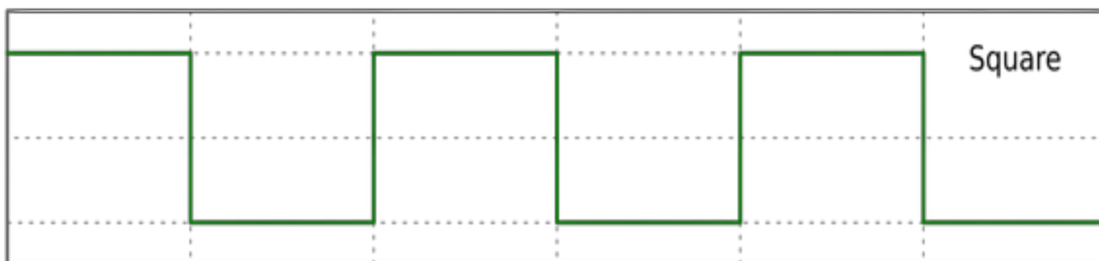
```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_Toggle;  
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable ;  
TIM_OCInitStructure.TIM_Pulse =10;  
TIM_OC2Init(TIM3,&TIM_OCInitStructure);  
TIM_ITConfig(TIM3,TIM_IT_CC2,ENABLE);  
GPIO_PinRemapConfig(GPIO_PartialRemap_TIM3, ENABLE);
```

TIM3	PA6、 PA7、 PB0、 PB1	PB4、 PB5、 PB0、 PB1		PC6、 PC7、 PC8、 PC9
------	-----------------------	-----------------------	--	-----------------------

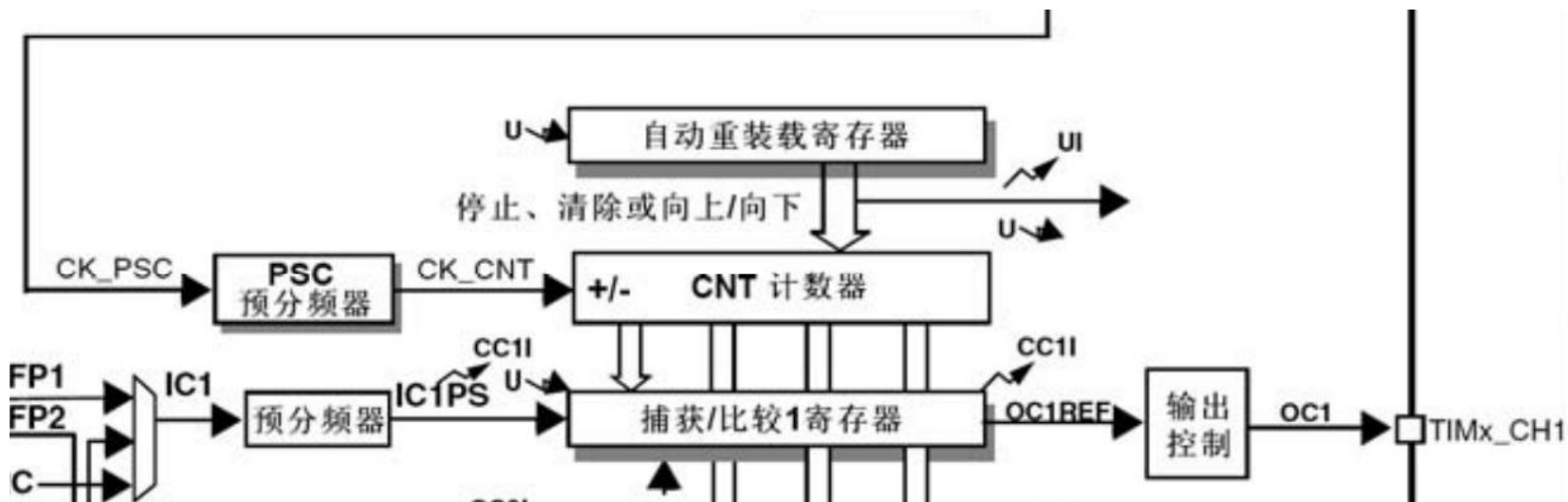
```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5 ;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

按键	频率	音符含义
1	262 Hz	低音/中音 Do
2	294 Hz	Re
3	329 Hz	Mi
4	349 Hz	Fa
5	392 Hz	Sol
6	440 Hz	La
7	494 Hz	Si
8	523 Hz	高音 Do
9	587 Hz	高音 Re

方波完整周期需要翻转两次：高电平 → 低电平 → 高电平



输出频率 = TIM3计数频率 / 比较间隔值 / 2
→ 比较间隔值 = TIM3计数频率 / 输出频率 / 2



比较间隔值: x

按键	频率	音符含义
1	262 Hz	低音/中音 Do
2	294 Hz	Re
3	329 Hz	Mi
4	349 Hz	Fa
5	392 Hz	Sol
6	440 Hz	La
7	494 Hz	Si
8	523 Hz	高音 Do
9	587 Hz	高音 Re

```
switch(key)
{
    case 1: x=500000/262/2; break;
    case 2: x=500000/294/2; break;
    case 3: x=500000/329/2; break;
    case 4: x=500000/349/2; break;
    case 5: x=500000/392/2; break;
    case 6: x=500000/440/2; break;
    case 7: x=500000/494/2; break;
    case 8: x=500000/523/2; break;
    case 9: x=500000/587/2; break;
    default:x=10;break;
}
```

```
TIM_TimeBaseStructure.TIM_Prescaler=144-1;  
TIM_TimeBaseStructure.TIM_Period=65535;  
TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;  
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;  
TIM_TimeBaseInit(TIM3, & TIM_TimeBaseStructure);  
TIM_Cmd(TIM3, ENABLE);
```

为什么 144-1 比较合适？

因为它让定时器计数频率变成：
 $500k\text{ Hz}$ ，这个频率对于电子琴
比较合适：低音时 CCR不会超过
65535；高音时 CCR也不会太小；
计算比较方便；音调精度比较好。

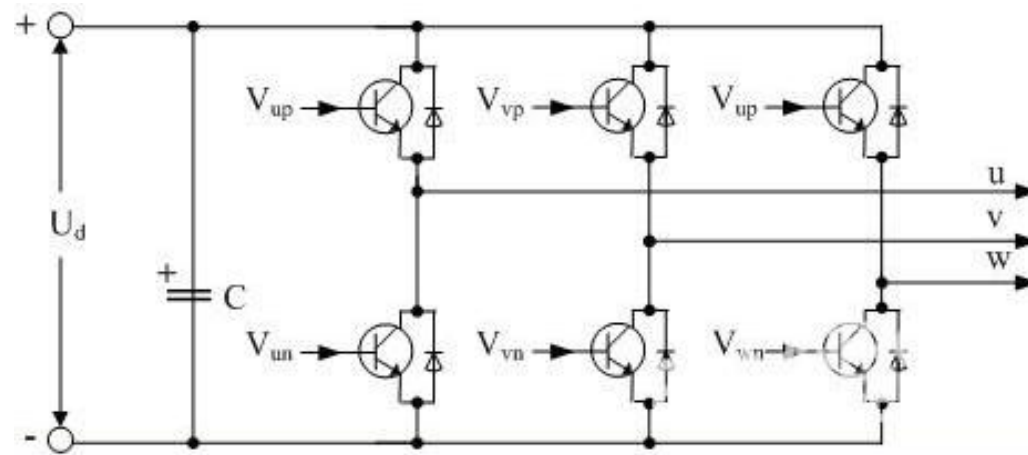
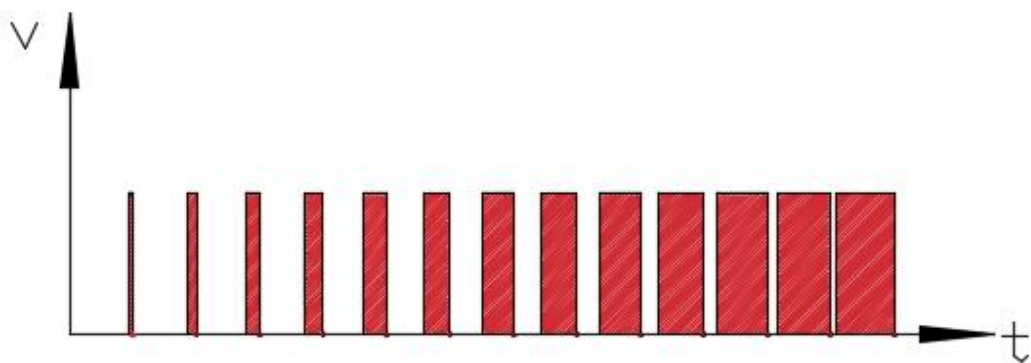
音符频率	CCR 约为
262 Hz	$500000/262/2 \approx 1908$
330 Hz	$500000/330/2 \approx 1514$
392 Hz	$500000/392/2 \approx 1274$
440 Hz	$500000/440/2 \approx 1135$
523 Hz	$500000/523/2 \approx 955$

1 2 3 1 | 1 2 3 1 |
3 4 5 - | 3 4 5 - |
5 6 5 4 3 1 | 5 6 5 4 3 1 |
1 5 1 - | 1 5 1 - |

练习：打开**自动播放**曲子-两只老虎

2.3 PWM模式

- PWM (Pulse Width Modulation)，脉宽调制信号，指**周期固定，脉冲宽度可以调节**的方波信号。在电机调速、变频器、DA转换电路中，经常使用。
- 对该信进行阻容滤波，可以得到模拟信号。
- 高级定时器TIM1，CH1~CH3各有互补的两个引脚，可以用来驱动**H桥逆变电路**。



逆变器，三相交流电

2.3 PWM模式

- 1、设置计数器的预分频系数

rescaler

、设置计数器的周期

period

。
- 2、设置某通道的比较值

pulse

，将该值放在该通道的捕捉比较寄存器中。
- 3、在计数器加法模式中，对于PWM1模式，比较成功

pulse

值，管脚置为低电平，计数器溢出

period

值时，管脚置为高电平。即当比较寄存器的值小于计数器的值时，通道引脚为高电平。
- 4、占空比为： D/P ，其中D为比较捕捉值

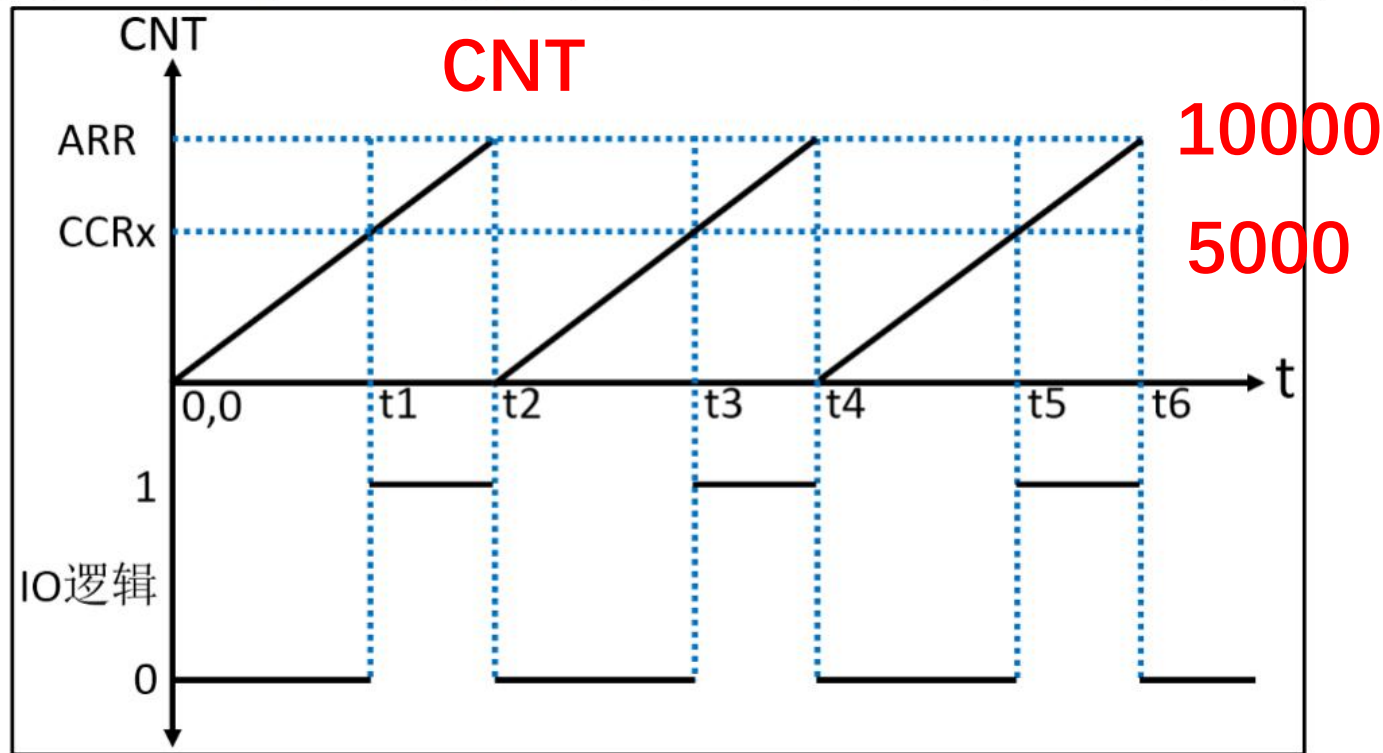
pulse

，P为周期

period

。
- 5、根据系统要求，可以随时修改比较值。
- 6、可以用示波器观察其波形。但示波器不方便，我们用LED灯，来做。
- 7、对于计数器的减模式和PWM2模式，大家可以自行研究，在此不赘述。

- 在计数器频率固定时，PWM 频率或者周期由自动重载寄存器（TIMx_ARR）的值决定，其占空比由捕获/比较寄存器（TIMx_CCRx）的值决定。



从图可以看到，当计数器的值 $CNT < CCRx$ 时，IO 输出低电平；当 $CNT > CCRx$ 时，IO 输出高电平；当 $CNT = ARR$ 时，定时器溢出，将 CNT 清零，依次循环。可以通过调整 $CCRx$ 的值，来调整占空比；通过改变 ARR 的值，可以调整 PWM 的频率。

```
TIM_TimeBaseStructure.TIM_Prescaler=7200-1;  
TIM_TimeBaseStructure.TIM_Period=10000-1; //PWM周期  
TIM_TimeBaseStructure.TIM_ClockDivision=TIM_CKD_DIV1;  
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up;  
TIM_TimeBaseInit(TIM4, & TIM_TimeBaseStructure);  
TIM_Cmd(TIM4, ENABLE);
```

```
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;  
TIM_OCInitStructure.TIM_OutputState =TIM_OutputState_Enable ;  
TIM_OCInitStructure.TIM_Pulse =5000; //CCR1,占空比50%  
TIM_OC1Init(TIM4,&TIM_OCInitStructure);
```

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);
```

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6| GPIO_Pin_7;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

例程：基于STM32F103 TIM4 单通道PWM 输出及 按键调节占空比的设计

- 1.初始化 STM32 系统时钟、GPIO、TIM4 和 SysTick;
- 2.使用 TIM4 通道 1 输出 PWM 波;
- 3.PWM 输出引脚为 PB6, 即 TIM4_CH1;
- 4.通过 3×3 矩阵键盘读取按键;
- 5.按下不同按键时, 改变 TIM4 通道 1 的比较值 CCR1;
- 6.通过改变 CCR1 改变 PWM 占空比;
- 7.用 8 位数码管动态显示当前 CCR1 的数值;
- 8.SysTick 每 1ms 中断一次, 用于数码管刷新和按键扫描。

TIM4

PB6、PB7、PB8、PB9

例程：基于STM32F103 TIM4 双通道PWM 输出及 按键调节占空比的设计

- 1.TIM4_CH1 和 TIM4_CH2 输出 PWM 波；
- 2.PWM 周期为 1 秒；
- 3.初始占空比为 50%；
- 4.通过 3×3 矩阵键盘调节 PWM 比较值 CCR；
- 5.通过改变 CCR 改变 PWM 占空比；
- 6.用 8 位数码管动态显示当前比较值；
- 7.SysTick 每 1ms 执行一次，用于数码管刷新和按键扫描。

TIM4

PB6、PB7、PB8、PB9